GROUP INFORMATION	
Group Number	5
System Owned	PUP-SPOT
Group Members	Daniel Cueto Jenver Napenas Aaron Paul Imperial Emmanuel Manansala Justine Philip Raguin

QA-001 – Poor Repository Organization

Affected Area: Repository folder structure, project organization, and documentation.

Affected Activity: Activity 3 – System Development and Repository Deployment; Activity 4 – Access Control Documentation.

Severity: Low–Medium

Status: Confirmed through repository inspection.

Description: The repository contains duplicate nested project directories, inconsistent folder naming conventions, and scattered documentation. The actual application is located under Helix-repo--main/Helix-repo--main/ instead of a clearly identified application root.

Evidence: Read-only inspection of the GitHub main branch confirmed the nested application structure, minimal root README, separate root user-role document, and inconsistent Activity 4 folder names such as G03_OJT, G2_ParkIt, and G7_CAMPUSAFE.

Expected Result: The repository should have a clearly identifiable application root, consistent folder naming conventions, and easily accessible documentation.

Actual Result: Application files are nested under duplicate directory names, the root README contains minimal information, and documentation is distributed across different directories.

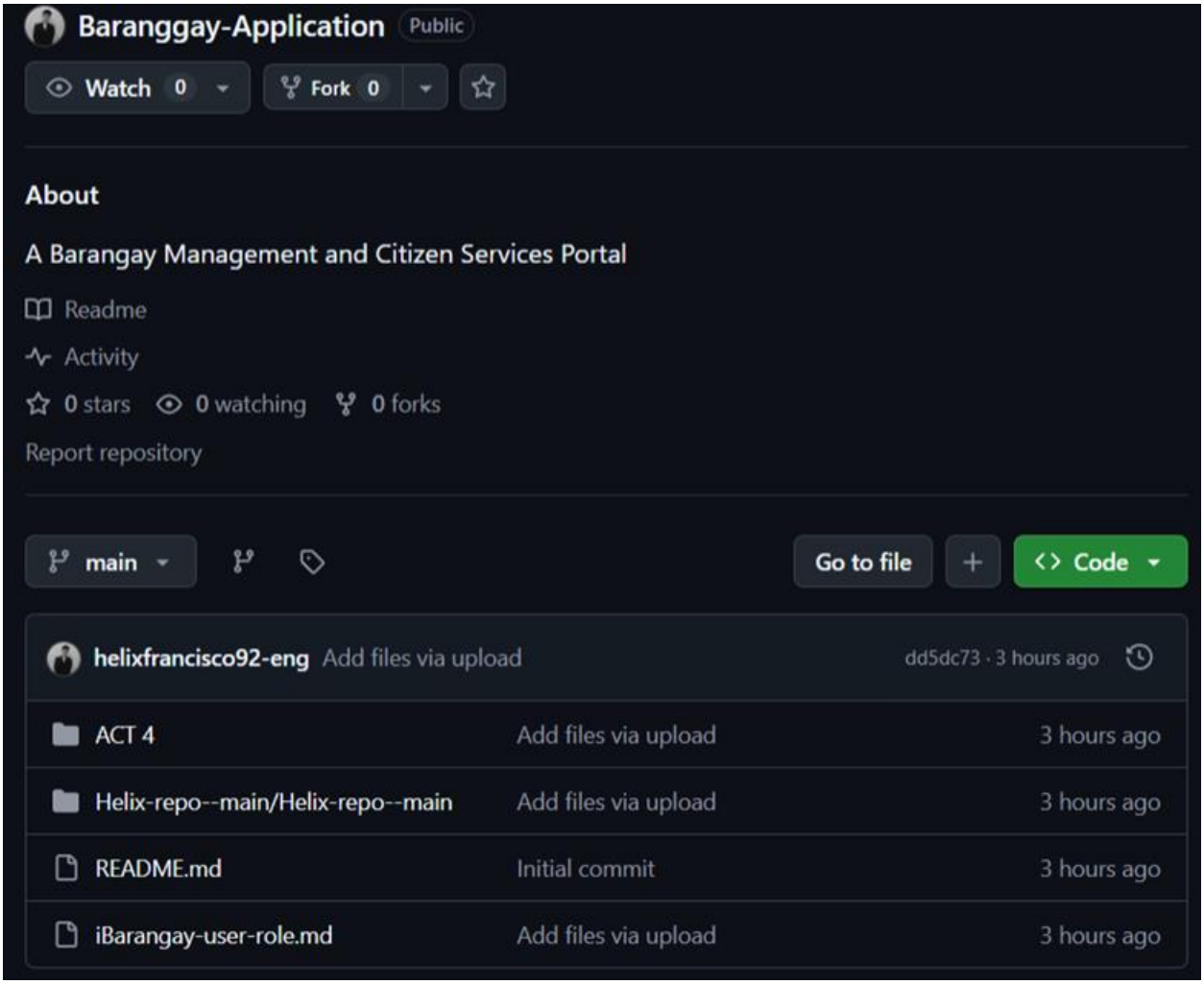
Impact: Increases difficulty navigating, configuring, maintaining, and testing the application. May cause setup confusion and unnecessary troubleshooting.

Recommended Fix: Consolidate the application under a clearly identified project root, improve the root README, standardize folder naming conventions, and organize documentation consistently.

Verification Method: GitHub repository tree and README inspection.

Test Result: Confirmed structural and documentation issues. No runtime failure was tested or claimed.

https://github.com/helixfrancisco92-eng/Baranggay-Application/tree/main



https://github.com/helixfrancisco92-eng/Baranggay-Application/tree/main/Helix-repo--main/Helix-repo--main



QA-002 – Unclear and Inaccurate Application Startup Instructions

Affected Area: README documentation, Windows launcher scripts, and database configuration.

Affected Activity: Activity 3 – System Development, Application Setup, and Repository Deployment.

Severity: Medium

Status: Confirmed documentation and configuration inconsistencies in inspected source files. A startup failure was not reproduced.

Description: The startup instructions contain a machine-specific path, describe two different launchers as interchangeable, reference an absent environment template, and do not clearly explain the relationship between launcher database variables and Spring Boot configuration.

Evidence: The GitHub root README contains only a short title and description. The nested README instructs users to run cd "E:\programming project\ibaranggay", states start.bat or setup_and_start.bat can create the database, and lists .env.example even though it is absent in the inspected main-branch tree. start.bat launches mvnw.cmd spring-boot:run without an explicit database-creation command, whereas setup_and_start.bat has CREATE DATABASE logic. setup_and_start.bat reads DB_NAME and DB_PORT while application.properties derives its JDBC URL from DB_URL. A separate setup_db.bat also exists. Earlier local-workspace Test-Path checks were reported; those outputs are not independently reproduced in this document.

Expected Result: A tester should be able to follow a clear, accurate, machine-independent path to configure the database and start the application.

Actual Result: The README and launcher behavior do not fully match, and the startup path is tailored to a particular machine.

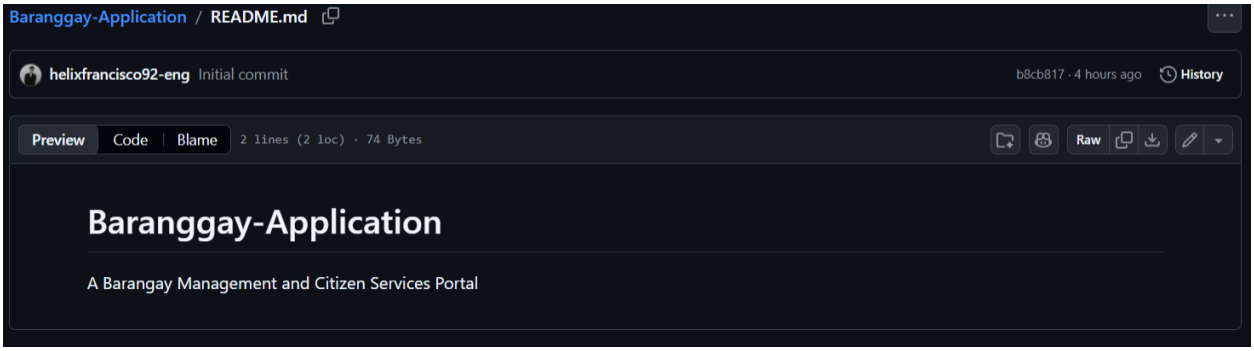
Impact: Unclear setup guidance may lead to misconfiguration, unnecessary troubleshooting, and database connection failures.

Recommended Fix: Put reliable quick-start instructions in the root README; use repository-relative paths; distinguish start.bat, setup_and_start.bat, and setup_db.bat; explain DB_URL versus DB_NAME/DB_PORT; add a safe .env.example; test with a fresh setup.

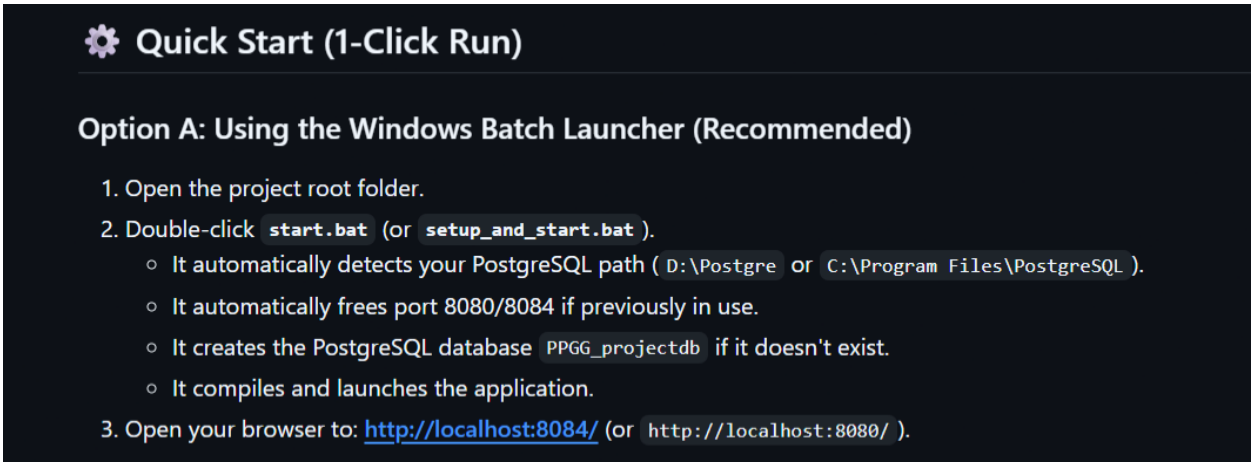
Verification Method: Read-only README, launcher, application.properties, and repository-tree inspection.

Test Result: Documentation and source discrepancies observed. No launch, database creation, or process termination was performed.

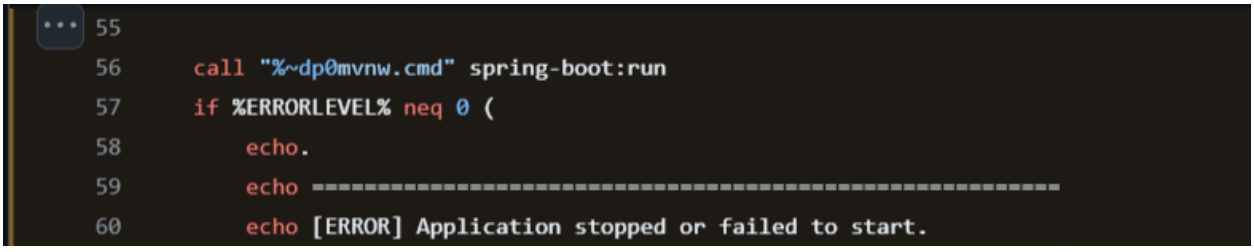
https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/README.md



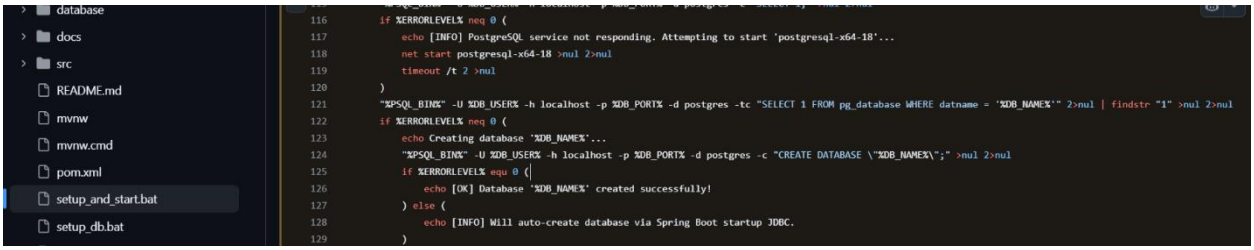
https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/README.md



https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/start.bat#L55-L60



https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/setup_and_start.bat#L115-L140



QA-003 – Machine-Dependent and Inflexible Setup Scripts

Affected Area: Windows setup/startup scripts, PostgreSQL configuration, and port-conflict handling.

Affected Activity: Activity 3 – System Development, Database Setup, and Application Deployment.

Severity: Medium

Status: Confirmed source-level limitations; cross-machine failures and process termination were not reproduced.

Description: Launchers depend on Windows-specific commands, search predefined PostgreSQL paths, attempt to start the fixed service postgresql-x64-18, and use partly inconsistent database settings. Two scripts forcibly terminate a PID found on their chosen port without checking whether it belongs to iBarangay.

Evidence: Recorded local read-only search: start.bat:30 and setup_and_start.bat:39 contain taskkill /F /PID %FOUND_PID%; setup_db.bat:80 and setup_and_start.bat:118 contain net start postgresql-x64-18; setup_and_start.bat reads DB_NAME and DB_PORT; application.properties:7 uses DB_URL. Both mvnw and mvnw.cmd exist. The GitHub files have also been inspected for these script behaviors. No launcher was executed.

Expected Result: Setup should specify supported platforms, honor consistent database settings, and handle occupied ports safely without automatically stopping an unidentified program.

Actual Result: Scripts use machine-dependent service recovery and forceful port cleanup; the application URL configuration is separate from launcher DB_NAME/DB_PORT.

Impact: Different installations may require manual work. Forced PID termination could stop unrelated local software if a conflicting port is occupied.

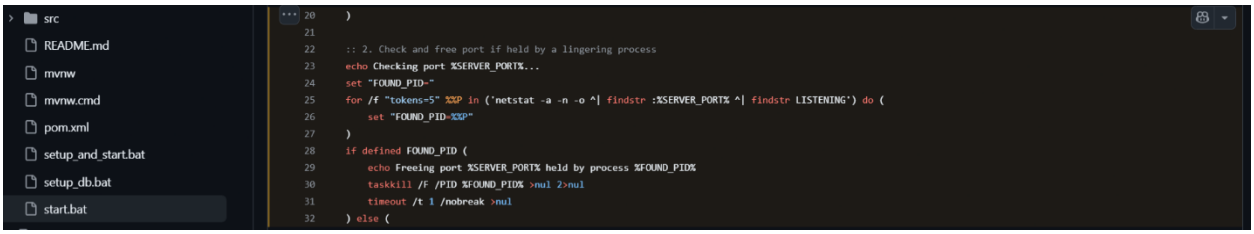
Recommended Fix: Document supported platforms and a manual route, make service/path handling configurable, standardize database configuration, and replace unconditional taskkill with a warning or an explicit user-selected action.

Verification Method: Static script inspection supported by local rg output and GitHub source review.

Test Result: Code-level risks observed; no alternate operating system test or live incident was performed.

Verification Limitation: The supplied evidence comes from a local workspace inspection. The remote main branch was not independently rechecked for this specific report.

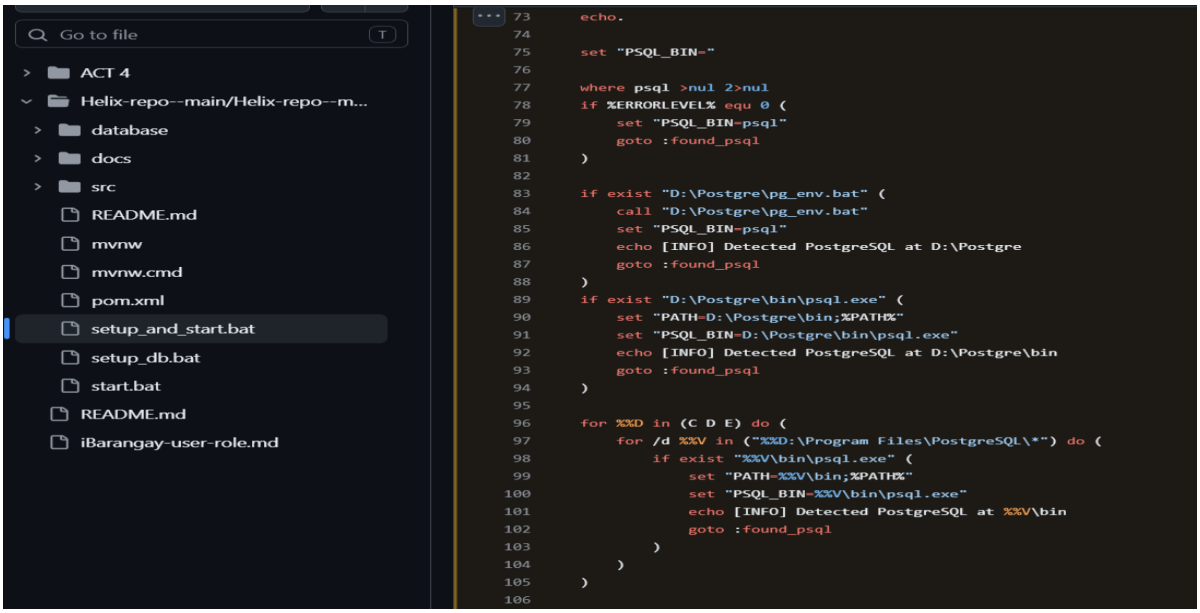
https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/start.bat#L20-L32



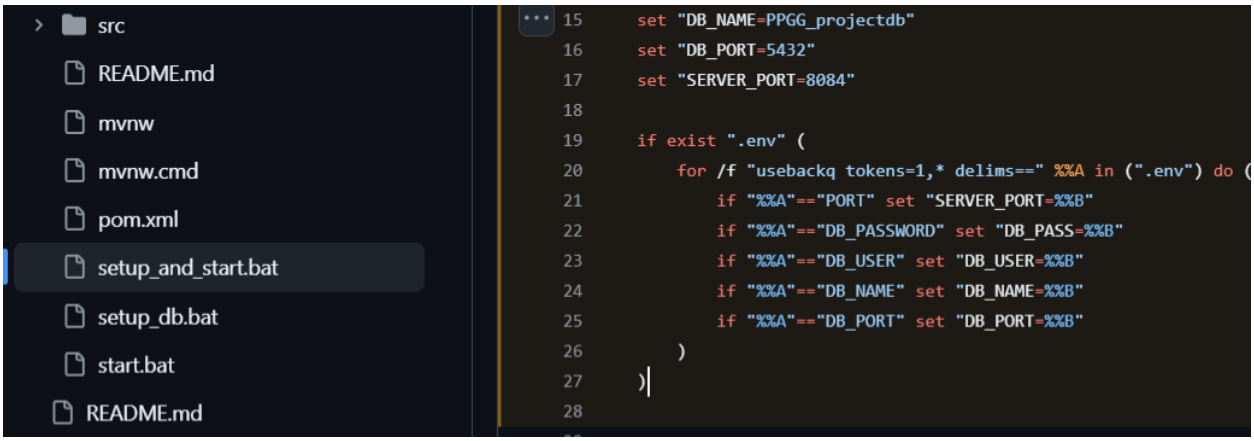
https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/setup_and_start.bat#L106-L121



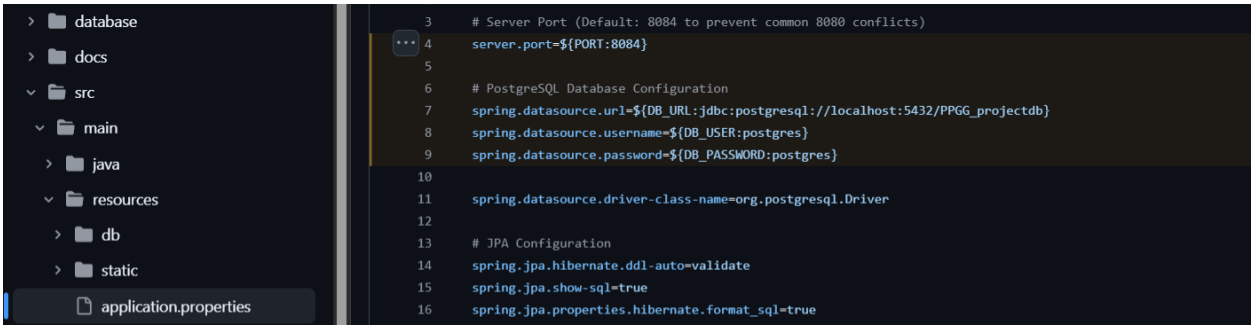
https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/setup_and_start.bat#L73-L106



https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/setup_and_start.bat#L15-L28



https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/src/main/resources/application.properties#L4-L9



QA-004 – Missing API Authentication and Authorization

Affected Area: Application security, administrative REST API, access control.

Affected Activity: Activity 3 – System Development and Application User Roles; Activity 4 – Access Control.

Severity: Critical (potential impact if API is reachable)

Status: Confirmed missing access checks in reviewed source; unauthorized runtime access not tested.

Description: Administrative endpoints allow creating, reading, updating, and deleting admin records without visible authentication or authorization checks in the reviewed application.

Evidence: AdminController.java exposes GET/POST/PUT/DELETE under /api/admins and calls AdminService repository operations without a user identity or role check. pom.xml does not list a Spring Security dependency. docs/user-roles.md defines administrator-only responsibilities.

Expected Result: Only authenticated, authorized administrators should modify administrative records.

Actual Result: No visible application-layer authentication or authorization gate in reviewed administrative controller/service.

Impact: If reachable, an unauthorized client may manipulate administrator records, undermining system control and integrity.

Recommended Fix: Implement authenticated sessions/tokens and enforce administrator-role authorization on the backend using Spring Security or equivalent. Add automated negative tests.

Verification Method: Static review of AdminController.java, AdminService.java, pom.xml, and role documentation.

Test Result: Control absent in reviewed implementation. No unauthorized test request was sent.

QA-005 – Unrestricted Access to Citizens' Personal Information

Affected Area: Application security, citizen API, data privacy.

Affected Activity: Activity 3 – Citizen Management; Activity 4 – Access Control and User Permissions.

Severity: Critical (potential confidentiality impact)

Status: Confirmed missing authorization at reviewed endpoints; external data exposure not tested.

Description: The citizens endpoints return full Citizen entities without visible identity, role, or record-level access checks.

Evidence: CitizenController.java exposes GET /api/citizens and GET /api/citizens/{citizenId} and returns Citizen objects. Citizen.java includes names, address, contact number, date of birth, and email. CitizenService.java retrieves entities through the repository. docs/user-roles.md describes more limited citizen access.

Expected Result: Only authorized staff should access complete citizen records; resident access should be limited to permitted data.

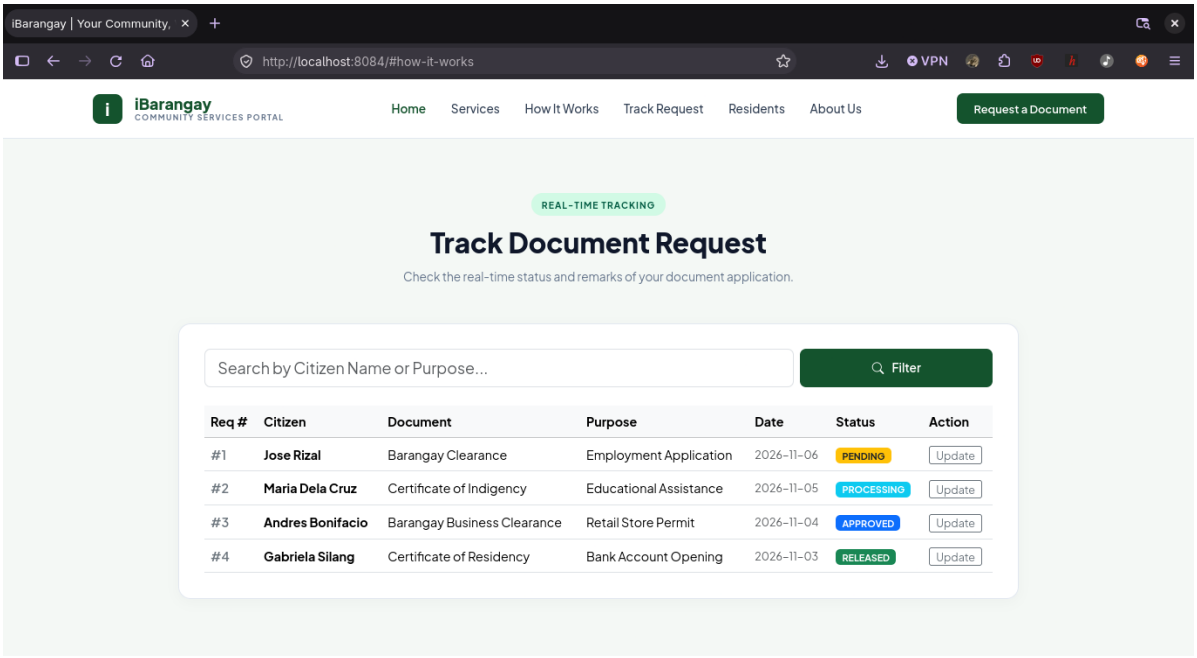
Actual Result: The reviewed GET handlers return citizen entities directly without visible access restrictions.

Impact: If endpoints are reachable, resident personal information could be disclosed to unauthorized clients.

Recommended Fix: Require authentication, role- and record-level authorization, and purpose-limited response DTOs; test forbidden requests and data minimization.

Verification Method: Static review of CitizenController.java, CitizenService.java, Citizen.java, and role documentation.

Test Result: Missing checks observed; no live personal-data retrieval was performed.


iBarangay | Your Community,
http://localhost:8084/#how-it-works
VPN
iBarangay
COMMUNITY SERVICES PORTAL
Home
Services
How It Works
Track Request
Residents
About Us
Request a Document
REAL-TIME TRACKING
Track Document Request
Check the real-time status and remarks of your document application.
Search by Citizen Name or Purpose...
Filter
Req #
Citizen
Document
Purpose
Date
Status
Action
#1
Jose Rizal
Barangay Clearance
Employment Application
2026-11-06
PENDING
Update
#2
Maria Dela Cruz
Certificate of Indigency
Educational Assistance
2026-11-05
PROCESSING
Update
#3
Andres Bonifacio
Barangay Business Clearance
Retail Store Permit
2026-11-04
APPROVED
Update
#4
Gabriela Silang
Certificate of Residency
Bank Account Opening
2026-11-03
RELEASED
Update

QA-006 – Unauthorized Modification and Deletion of Document Requests

Affected Area: Document request APIs, data integrity, application authorization.

Affected Activity: Activity 3 – Document Request Processing; Activity 4 – Role-Based Access Control.

Severity: Critical (potential integrity impact)

Status: Confirmed missing checks in reviewed code; unauthorized modification not reproduced.

Description: The application exposes modification and deletion operations for document requests without visibly requiring an authenticated, appropriately privileged operator.

Evidence: DocumentRequestController.java exposes PUT and DELETE /api/document-requests/{requestId}. Its update code accepts status, remarks, purpose, processedAt, releasedAt, and approvedBy fields and delegates saving to DocumentRequestService.java. No role verification appears in these handlers. docs/user-roles.md assigns processing responsibilities to staff/admin.

Expected Result: Only authorized barangay staff should approve, reject, release, or delete requests, under a controlled workflow.

Actual Result: Sensitive document-request fields are updated or deleted in reviewed handlers without visible authorization.

Impact: Unauthorized users could potentially falsify status/approval fields or remove official request records.

Recommended Fix: Require authenticated backend role checks; restrict sensitive fields; enforce request-state transitions; record a transactionally consistent audit history.

Verification Method: Static review of DocumentRequestController.java, DocumentRequestService.java, DocumentRequest.java, and roles documentation.

Test Result: Missing authorization checks observed. No unauthorized update or delete operation was run.

QA-007 – Missing Implementation of Application Role Separation

Affected Area: Application architecture, documented roles, backend RBAC; frontend organization is contextual.

Affected Activity: Activity 3 – Application User Roles and System Development; Activity 4 – Access Control Configuration.

Severity: Critical (shares root cause with QA-004; do not double-count as independent exploit)

Status: Confirmed mismatch between documented roles and reviewed application-layer enforcement. Frontend role experience remains only partly evaluated.

Description: docs/user-roles.md defines Barangay Administrator, Staff, Auditor, and Citizen duties, but the inspected API controllers do not visibly enforce those distinctions. Frontend interface separation alone is not a security control and is not required if backend permissions are correctly enforced.

Evidence: docs/user-roles.md describes role-specific functions. AdminController.java, CitizenController.java, and DocumentRequestController.java lack visible authentication/role checks; pom.xml has no Spring Security dependency. A single static index.html and HomeController.java exist, but a shared frontend does not itself prove a vulnerability.

Expected Result: Backend operations must verify user identity and enforce allowed operations and records for each documented role; UI should present role-appropriate functionality.

Actual Result: Documented role rules are not demonstrably enforced by the reviewed application API.

Impact: Users may reach operations outside their documented role boundaries, depending on network access and deployment.

Recommended Fix: Implement authenticated role mapping and backend authorization by endpoint and record; test Admin, Staff, Auditor, and Citizen positive/negative permissions. Adjust frontend navigation to roles where appropriate.

Verification Method: Static comparison of docs/user-roles.md with controllers, dependencies, and frontend entry points.

Test Result: Document-to-implementation gap observed; no role-bypass runtime test. Overlaps QA-004 rather than representing a separate proven compromise.

https://github.com/helixfrancisco92-eng/Baranggay-Application/blob/main/Helix-repo--main/Helix-repo--main/src/main/resources/application.properties

```
# PostgreSQL Database Configuration
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/PPGG_projectdb}
spring.datasource.username=${DB_USER:postgres}
spring.datasource.password=${DB_PASSWORD:postgres}
```

QA-008 – No Documented Automated Database Backup Procedure

Affected Area: Database administration, backup management, data recovery, and project documentation.

Affected Activity: Activity 3 – Database Setup and Documentation; Activity 5 – Database Maintenance and Reliability.

Severity: High

Status: Confirmed missing backup documentation and identifiable backup scripts in the inspected repository. Actual backup availability and restoration capability remain unverified.

Description: The repository does not provide a clearly documented automated database backup procedure, backup schedule, retention policy, or restoration process. No identifiable backup or restoration scripts were found in the inspected repository structure.

Evidence: Read-only inspection of the GitHub main branch, including the database directory, README, database documentation, progress documentation, and troubleshooting guide. No backup-related procedures were identified. See screenshots QA-008-Evidence-01 through QA-008-Evidence-03.

Expected Result: The project should document how PostgreSQL backups are generated, scheduled, stored securely, retained, and restored. Backup restoration should be periodically verified in an authorized test environment.

Actual Result: The inspected repository does not document an automated backup schedule, retention policy, or recovery procedure. No recent backup or successful restoration test was available for verification.

Impact: If a database failure, accidental deletion, or data corruption occurs, recovery may be delayed or unsuccessful if usable backups are not available.

Recommended Fix: Establish an automated PostgreSQL backup procedure, define backup frequency and retention rules, securely store backup files separately from the live database, and document a restoration procedure. Periodically test recovery using an authorized non-production database.

Verification Method: Read-only inspection of repository files and project documentation. No database operations were performed.

Test Result: Missing repository-level backup documentation confirmed. Actual backup existence, scheduling, and recovery success have not been tested.

“Refer to the QA-002 – Unclear and Inaccurate Application Startup Instructions screenshot”

QA-009 – Excessive Database Queries When Retrieving Document Requests

Affected Area: GET /api/document-requests, DocumentRequestController, DocumentRequestService, DocumentRequestRepository, and DocumentRequest entity relationships.

Affected Activity: Activity 5 – Database Performance Monitoring and Query Optimization.

Severity: Medium

Status: Reported as confirmed through manual reproduction using Hibernate SQL logging. The supplied results have not been independently reverified from the original execution logs.

Description: The application uses the default findAll() repository method to retrieve document requests. The DocumentRequest entity defines its relationships with Citizen, DocumentType, and Admin as lazy-loaded. When these relationships are accessed during JSON serialization, Hibernate executes additional queries to retrieve associated records, resulting in an N+1 query problem.

Evidence: Source-code inspection identified that DocumentRequestController.getAll() calls DocumentRequestService.getAllDocumentRequests(), which uses documentRequestRepository.findAll(). The DocumentRequest entity defines three FetchType.LAZY relationships. According to the supplied Hibernate SQL logging results, the original implementation executed 11 SQL statements: one query for document requests and ten additional entity-fetch queries. A custom repository query using JOIN FETCH for citizen, documentType, and approvedBy reportedly retrieved the required relationships using one SQL statement.

Expected Result: Document requests and the related information required by the API response should be retrieved efficiently without unnecessary database queries.

Actual Result: The original implementation generated 11 SQL statements, while the tested fetch-join implementation generated one statement for the same required relationships.

Impact: Causes unnecessary database queries, increasing database workload and processing overhead. Performance may deteriorate as the number of document requests and related records increases.

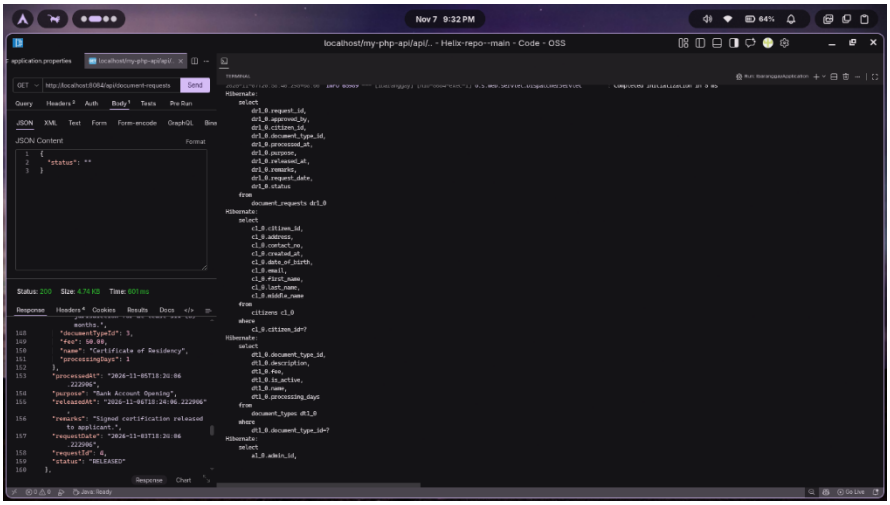
Recommended Fix: Use a custom repository query with JOIN FETCH or @EntityGraph to retrieve the required relationships efficiently. Alternatively, use a DTO projection to retrieve only the necessary response information. Validate the optimization using Hibernate SQL logs and representative test datasets.

Verification Method: Manual source-code inspection, Hibernate SQL logging, and GET requests to /api/document-requests. The original findAll() implementation was compared against a custom JOIN FETCH implementation.

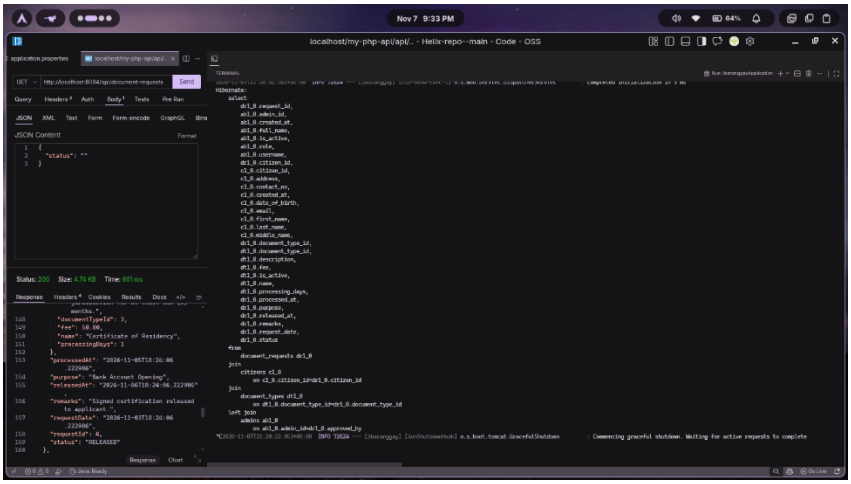
Test Result: According to the supplied manual test, the original implementation executed 11 SQL statements, while the optimized implementation executed one SQL statement, a reduction of approximately 90.9%. The required related records were reportedly retrieved, and no database records were modified.

Verification Limitation: Original command output or Hibernate SQL log screenshots were not provided for independent review. The reported results describe the tested environment and should not be interpreted as a guaranteed improvement for every workload.

Hibernate SQL Logs Showing Repeated Queries in the Original findAll() Implementation (11 Queries in total. not captured all 11 in terminal)



Hibernate SQL Logs Showing Single-Query Retrieval Using JOIN FETCH



GROUP INFORMATION	
Group Number	5
System Owned	PUP-SPOT
Group Members	Daniel Cueto Jenver Napenas Aaron Paul Imperial Emmanuel Manansala Justine Philip Raguin

INC-01 – Setup Script Fails to Adapt to Different Environments

Incident ID and Related Flaw: INC-01; QA-003 – Machine-Dependent and Inflexible Setup Scripts.

Affected Area: Windows setup scripts, PostgreSQL installation/service configuration, and database connection settings.

Affected Activity: Activity 3 – System Development, Database Setup, and Application Deployment.

Difficulty (Design Sheet): Easy; estimated 15–20 minutes (planned, not a measured duration).

Status: Confirmed source-code limitations; cross-machine startup failure has not been reproduced.

Incident Brief: On a machine with a different PostgreSQL installation directory or service name, the project's startup process may fail to initialize the expected database connection.

Scenario: A tester launches the application on a Windows computer where PostgreSQL is installed outside the predefined search locations or where its Windows service is named differently. The launchers attempt environment-specific recovery and may not establish a working database connection.

Evidence: QA-003 identifies fixed service references in setup_db.bat and setup_and_start.bat (postgresql-x64-18); predefined installation-path checks; different DB_NAME/DB_PORT launcher settings versus the DB_URL used by application.properties; and forceful port cleanup in start.bat and setup_and_start.bat. See the QA-003 source screenshots and recorded read-only rg output in the flaw document.

Expected Root Cause: Machine-dependent service/path assumptions and inconsistent configuration sources. This is the expected cause of the proposed scenario, not a proven cause of a live outage.

Recommended Fix: Document supported operating systems and a manual setup procedure; make PostgreSQL service/path settings configurable; consistently resolve the database connection URL; replace automatic termination of unrelated processes with safe port-conflict handling.

Verification Method: On an authorized test computer, configure PostgreSQL with a different service name or directory. Confirm connectivity, successful startup, and safe behavior when the application port is occupied.

Test Result / Limitation: The scripts were inspected without being run. No alternate-machine setup failure, PostgreSQL modification, or process termination was observed. Fault and reset scripts remain placeholders in the incident design sheet.

INC-02 – Lack of Separation Between Citizen and Administrative Application Functions

Incident ID and Related Flaws: INC-02; QA-007 – Missing Implementation of Application Role Separation; related QA-004 (missing API authentication) and QA-006 (unrestricted request modification).

Affected Area: Application-layer identity checks, citizen/admin APIs, documented role permissions, and interface access.

Affected Activity: Activity 4 – Application and Database Access-Control Review (drawing on Activity 3 roles documentation).

Difficulty (Design Sheet): Hard; estimated 25–30 minutes (planned).

Status: Role-enforcement gap confirmed by static source review; unauthorized runtime access not tested.

Incident Brief: Documented Citizen, Staff, Auditor, and Administrator responsibilities are not visibly enforced by the reviewed API endpoints. A citizen-facing client could potentially reach administrative operations if those endpoints are exposed.

Scenario: A user intended to access only citizen services attempts to manage administrator records or change document requests. The test checks whether authentication and role-based backend controls reject unauthorized requests.

Evidence: docs/user-roles.md distinguishes the application roles; AdminController.java exposes administrative CRUD endpoints, while CitizenController.java and DocumentRequestController.java expose other operations without visible identity/role checks. pom.xml does not list Spring Security. The presence of a shared index.html alone does not establish a security violation.

Expected Root Cause: Missing application-level authentication and role-based authorization enforcement; the documented role rules are not demonstrably applied to the inspected backend. This overlaps QA-004 and is not a separate proven exploit.

Recommended Fix: Implement authenticated user identities and backend authorization for each role and sensitive action; use record-level checks for citizen records and document requests; present role-appropriate interface controls without relying on the UI for security.

Verification Method: In an authorized local environment with synthetic records, exercise the APIs as Citizen, Staff, Auditor, and Administrator. Record positive and denied operations; confirm citizens cannot access admin-only functions and authorized administrators still can.

Test Result / Limitation: Source inspection supports the missing controls. No role-bypass request or exploitation test was performed. Fault and reset scripts remain placeholders in the incident design sheet.

INC-03 – No Automated Database Backup Procedure

Incident ID and Related Flaw: INC-03; QA-007 - No Documented Automated
Database Backup Procedure.

Affected Area: Database backups, recovery procedures, scheduling, retention, and maintenance documentation.

Affected Activity: Activity 6 – Backup and Recovery (as assigned in the original incident design sheet); the QA report also references Activities 3 and 5.

Difficulty (Design Sheet): Medium; estimated 15–20 minutes (planned).

Status: Missing repository-level documentation confirmed; real backup availability and restore capability are unknown.

Incident Brief: The inspected repository does not explain how regular PostgreSQL backups are scheduled, retained, or restored. If the database is lost or corrupted, the team may not have a documented recovery path.

Scenario: After a simulated database outage, a tester seeks a recent usable backup and attempts to restore it into a separate non-production database. The scenario checks whether a reliable recovery procedure actually exists.

Evidence: QA-008 reports no identifiable backup/restore scripts or documented backup schedule, retention policy, or recovery instructions in the reviewed database directory, README, database documentation, progress documentation, and troubleshooting guide. No backup file or restore output was supplied.

Expected Root Cause: Undocumented or absent backup automation. The absence of repository instructions does not establish that no external backup exists.

Recommended Fix: Document and schedule consistent PostgreSQL backups (for example using pg_dump where appropriate), secure storage and retention rules, ownership/responsibility, and a step-by-step restore playbook.

Verification Method: With authorized test data, create a backup, restore to a separate test database, and compare expected tables and record counts. Record backup timestamps, backup location, retention behavior, and restore evidence without exposing private records or secrets.

Test Result / Limitation: Repository-level backup documentation gap is verified. No live backup/restore operation was carried out; actual restore readiness remains unverified. Fault and reset scripts were placeholders in the incident design sheet.

INC-04 – Missing Database Role Separation and Least-Privilege Permissions

Incident ID and Related Flaw: INC-04; no directly confirmed PostgreSQL privilege flaw in the current QA-001–QA-009 documentation. QA-004 and QA-007 relate to APPLICATION-layer authorization only.

Affected Area: PostgreSQL login roles, role memberships, schema/table privileges, and configured application runtime credentials.

Affected Activity: Activity 4 – Database Privileges and Role Management.

Difficulty (Design Sheet): Medium; estimated 10-20 minutes (planned).

Status: Open investigation; excessive database privileges have not been established by current evidence.

Incident Brief: Investigate whether the database runtime account has broader access than its job requires and whether database administration is properly separated from ordinary application activity.

Scenario: A database account used for routine application functions attempts an administrative database action. The investigation compares actual grants and permissions with the documented Activity 4 roles and expected least-privilege rules.

Evidence: The incident sheet identifies migration/setup SQL, application.properties, database configuration source, PostgreSQL role memberships, and grants as necessary sources. The flaw document establishes weak application API authorization, but it does not include a completed database privilege matrix or role-membership test proving a PostgreSQL grant violation.

Expected Root Cause: Potentially excessive runtime database permissions or missing separation of DBA and application accounts; this remains a hypothesis pending database verification.

Recommended Fix: Inventory actual database accounts, roles, default privileges, schema/table/sequence grants and current application credentials. Only after confirming excessive access, create or adjust least-privilege roles with an authorized administrator.

Verification Method: Use safe read-only catalog/privilege inspections and role-appropriate negative/positive tests on a test database. Verify intended operations succeed and unauthorized operations fail. Avoid exposing passwords or changing shared cluster permissions without authorization.

Test Result / Limitation: No verified database-level violation or corrective action is documented. The incident is an investigation scenario, not a confirmed occurrence. Fault/reset scripts remain placeholders.

INC-05 – Excessive Database Queries When Retrieving Document Requests

Incident ID and Related Flaw: INC-05; QA-009 – Excessive Database Queries When Retrieving Document Requests. (Corrected the source incident sheet's duplicate INC-04 label.)

Affected Area: GET /api/document-requests, DocumentRequestController, DocumentRequestService, DocumentRequestRepository, Hibernate entity loading.

Affected Activity: Activity 5 – Database Performance Monitoring and Query Optimization.

Difficulty (Design Sheet): Easy; estimated 5–10 minutes (planned).

Status: Manual reproduction reported in the QA document; full original Hibernate log sequence was not independently supplied.

Incident Brief: The document request endpoint loads related entities through multiple SQL statements when default findAll() results and lazy relationships are serialized, increasing database activity.

Scenario: As document requests grow, list retrieval repeatedly queries citizen, document-type, and administrator entities. The endpoint still returns records, but it may perform unnecessary SQL work.

Evidence: QA-009 identifies DocumentRequestService.getAllDocumentRequests() calling repository.findAll(), with Citizen, DocumentType, and Admin relationships marked FetchType.LAZY. Reported Hibernate logging found 11 SQL statements for the original path versus one for a custom JOIN FETCH implementation. The QA document includes illustrative log screenshots, but states that not all 11 original queries were captured.

Expected Root Cause: N+1 query behavior arising from default findAll() combined with lazy entity access during response serialization.

Recommended Fix: Use a targeted JOIN FETCH query, @EntityGraph, or DTO projection for required response fields; avoid fetching unrelated data; compare SQL counts, plans and response correctness before adopting an optimization.

Verification Method: Using a disposable local test setup, compare Hibernate SQL logging for the original and optimized GET endpoint, verify equivalent required response fields, and repeat on representative datasets. Preserve original records.

Test Result / Limitation: According to QA-009, original: 11 SQL statements; optimized: 1 statement (reported 90.9% reduction). Complete raw log output and performance under scale were not independently verified. The incident sheet states no fault/reset scripts are required; changes should be tested on a disposable copy.

Group 05 — Findings and Owner Responses (Supplied Image)

			transactions and their details.	split tender payments.	payment method used.	amounts are rejected.
G05-INC1	G05	G11	In src/controllers/adminController.php, the generic lost-report route is placed before the image route, causing the router to misinterpret image URLs as invalid report IDs.	The finding is valid. The generic lost-report route was placed before the image route, causing image requests to be incorrectly interpreted as report ID requests. We will correct this by moving the image route above the generic route to ensure attached photos can be accessed properly.	Administrators cannot view attached photos of lost reports in the admin dashboard because requests fail with an HTTP 404 "Resource not found" error.	The finding was valid. The route-ordering issue in `src/controllers/adminController.php` has been fixed by moving the lost-report image route above the generic lost-report route. Administrators can now access attached lost-report photos through the correct endpoint.
G05-INC2			In src/services/claimService.php, approving a claim updates the claim record but never updates the associated item's status to claimed.	This behavior is intentional and follows the system's designed workflow. Approving a claim does not automatically mark an item as claimed because approval does not guarantee that the claimant will actually show up to retrieve it. The item is only marked as claimed when the claimant successfully retrieves it, at which point the administrator manually updates its status. Automatically changing the status upon approval could incorrectly make an item unavailable even if the claimant never shows up. Therefore, this finding appears to be a misinterpretation of the intended workflow rather than an actual system flaw.	Already approved items remain displayed as unclaimed in the public catalog, allowing other users to submit duplicate claims for the same item.	The finding was determined to be a misinterpretation of the intended workflow. Claim approval does not automatically mean the item has been physically retrieved. The administrator only updates the item's status to `claimed` after the claimant successfully collects it. This prevents items from being incorrectly marked as unavailable when an approved claimant fails to show up. Therefore, no system changes were necessary.
G05-INC3			In public/app.html, the image file input enforces a mandatory required attribute, directly contradicting the API documentation and backend service where image uploads are optional.	The finding is valid. The image upload was marked as required in the frontend HTML and was also enforced by JavaScript validation, despite image uploads being optional in the backend and API documentation. We will correct both validations to allow lost reports to be submitted without photos.	Students who lost an item without having a prior photo of it are blocked by browser validation and cannot submit a lost report.	The finding was valid. The `required` attribute was removed from the image input in `public/app.html`, and the JavaScript validation in `public/js/app.js` was updated to allow lost reports without image attachments. Image requirements for claim submissions remain unchanged.
G05-INC4			The documentation in docs/README.md asserts that the public entry point uses the admin database connection, contradicting public/index.php which actively uses the restricted public connection.	Acknowledged outdated README file. Will update	Auditors and developers face conflicting information between the documented security architecture and the actual least-privilege connection used by the system.	The finding was valid. The documentation contained outdated information about the public database connection. It has since been updated to reflect the use of getPublicConnection().
G05-INC5			In src/controllers/publicController.php, reference codes are read from request bodies without existence checks, passing null into strictly typed PHP 8 string parameters.	The finding is valid. The same missing input validation issue also exists in the `/api/public/claims/view` endpoint, where a missing or invalid `claim_code` can cause a server error. We will address this by adding input validation and returning an appropriate HTTP 400 response for invalid requests.	Submitting an empty or malformed JSON request to the tracking endpoints crashes the server with an unhandled fatal TypeError (HTTP 500) and raw HTML.	The finding was valid. The missing input validation in the lost report endpoints has been fixed. The same issue was also identified in /api/public/claims/view and corrected to return an HTTP 400 JSON response for invalid or missing claim codes.

Contribution Record

The following record summarizes the responsibilities undertaken by the Group 05 members for the work documented in this report.

Group member	Contribution
Jenver Napenas	Documentation of reports and findings.
Aaron Paul Imperial	Identifying and reviewing potential issues in the project.
Daniel Cueto	Testing and validating the identified issues.
Emmanuel Manansala	Identifying and reviewing potential issues in the project.
Justine Philip Raguin	Documentation of reports and findings.

AI Usage Declaration

AI tools were used to help speed up the initial process of identifying and reviewing possible issues in the project. The group did not automatically accept AI-generated suggestions as confirmed flaws. Each proposed issue was reviewed and validated by the members against the project's source code and available documentation before it was treated as a finding.

AI tools were also used to help organize and document the findings in clear, readable report format. The group reviewed the resulting descriptions and documentation to ensure that they accurately represented the issues identified and the actions taken.

Validation statement: AI assistance was used as a support tool for issue discovery and report preparation; the group remained responsible for checking the findings and confirming the reported results.